2D CNN MODEL

```
# 2D CNN MODEL

# Imports all required libraries

import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, roc_curve
from sklearn.utils.class_weight import compute_class_weight

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (
    Conv2D,
    Dense,
    Dropout,
    Input,
    BatchNormalization,
    MaxPooling2D,
    GlobalAveragePooling2D
)
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.regularizers import l2


# Upload required datasets (training data, validation data and test data)

from google.colab import files
uploaded = files.upload()


# Auto detect the uploaded files

files_list = os.listdir()

def find_file(keyword):
    matches = [f for f in files_list if keyword in f.lower()]
    return matches[0]

train_file = find_file("augmented")
val_file   = find_file("validation")
test_file  = find_file("test")

print("Train:", train_file)
print("Val  :", val_file)
print("Test :", test_file)


# Load data

train_df = pd.read_csv(train_file)
val_df   = pd.read_csv(val_file)
test_df  = pd.read_csv(test_file)


# Split

X_train = train_df.iloc[:, :-1].values
y_train_raw = train_df.iloc[:, -1].values

X_val = val_df.iloc[:, :-1].values
y_val_raw = val_df.iloc[:, -1].values

X_test = test_df.iloc[:, :-1].values
y_test_raw = test_df.iloc[:, -1].values


# Class weights

classes = np.unique(y_train_raw)
weights = compute_class_weight(
```

```
    class_weight='balanced',
    classes=classes,
    y=y_train_raw
)
class_weights = dict(zip(classes, weights))


# Convert labels to one hot encoding

y_train = to_categorical(y_train_raw)
y_val   = to_categorical(y_val_raw)
y_test  = to_categorical(y_test_raw)


# Reshape data for 2D CNN input
height = 5
width = 4

padding = (height * width) - X_train.shape[1]

X_train = np.pad(X_train, ((0, 0), (0, padding)), mode='constant')
X_val   = np.pad(X_val, ((0, 0), (0, padding)), mode='constant')
X_test  = np.pad(X_test, ((0, 0), (0, padding)), mode='constant')

X_train = X_train.reshape((X_train.shape[0], height, width, 1))
X_val   = X_val.reshape((X_val.shape[0], height, width, 1))
X_test  = X_test.reshape((X_test.shape[0], height, width, 1))

print("Shape:", X_train.shape)


# Regulate overfitting

model = Sequential([

    Input(shape=(height, width, 1)),

    Conv2D(
        16,
        (2, 2),
        padding='same',
        activation='relu',
        kernel_regularizer=l2(1e-3)
    ),
    BatchNormalization(),

    Conv2D(
        32,
        (2, 2),
        padding='same',
        activation='relu',
        kernel_regularizer=l2(1e-3)
    ),
    BatchNormalization(),

    MaxPooling2D((2, 2)),
    Dropout(0.4),

    Conv2D(
        64,
        (2, 2),
        padding='same',
        activation='relu',
        kernel_regularizer=l2(1e-3)
    ),
    BatchNormalization(),

    GlobalAveragePooling2D(),

    Dense(
        64,
        activation='relu',
        kernel_regularizer=l2(1e-3)
    ),
    Dropout(0.5),

    Dense(
        32,
        activation='relu',
        kernel_regularizer=l2(1e-3)
    ),
    Dropout(0.4),
```

```
    Dense(
        y_train.shape[1],
        activation='softmax'
    )
])


# Compile model

model.compile(
    optimizer=Adam(learning_rate=0.0005),
    loss=tf.keras.losses.CategoricalCrossentropy(label_smoothing=0.1),
    metrics=['accuracy']
)

model.summary()


# Callbacks

early_stop = EarlyStopping(
    monitor='val_loss',
    patience=6,
    restore_best_weights=True
)

reduce_lr = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.3,
    patience=3,
    min_lr=1e-6,
    verbose=1
)


# Train model

history = model.fit(
    X_train,
    y_train,
    epochs=80,
    batch_size=64,
    validation_data=(X_val, y_val),
    callbacks=[early_stop, reduce_lr],
    class_weight=class_weights,
    verbose=1
)


# Evaluation

y_pred = model.predict(X_test)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true = y_test_raw

accuracy = accuracy_score(y_true, y_pred_classes)
precision = precision_score(y_true, y_pred_classes, average='macro')
recall = recall_score(y_true, y_pred_classes, average='macro')
f1 = f1_score(y_true, y_pred_classes, average='macro')

print("\n2D CNN PERFORMANCE")
print(f"Accuracy  : {accuracy:.4f}")
print(f"Precision : {precision:.4f}")
print(f"Recall    : {recall:.4f}")
print(f"F1 Score  : {f1:.4f}")


# Confusion Matrix

cm = confusion_matrix(y_true, y_pred_classes)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix")
plt.show()


# ROC Curve

plt.figure(figsize=(8, 6))
```

```
for i in range(y_test.shape[1]):
    fpr, tpr, _ = roc_curve(y_test[:, i], y_pred[:, i])
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, label=f'Class {i} (AUC={roc_auc:.2f})')

plt.plot([0, 1], [0, 1], 'k--')
plt.title("ROC Curve")
plt.legend()
plt.show()


# Training curves

plt.plot(history.history['accuracy'], label='Train')
plt.plot(history.history['val_accuracy'], label='Val')
plt.legend()
plt.title("Accuracy")
plt.show()

plt.plot(history.history['loss'], label='Train')
plt.plot(history.history['val_loss'], label='Val')
plt.legend()
plt.title("Loss")
plt.show()


# Save model

model.save("cnn_2d_model.keras")
print("\nModel saved successfully")
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving test_set_scaled.csv to test_set_scaled.csv
Saving validation_set_scaled.csv to validation_set_scaled.csv
Saving preprocessed_augmented_dataset_scaled.csv to preprocessed_augmented_dataset_scaled.csv
Train: preprocessed_augmented_dataset_scaled.csv
Val  : validation_set_scaled.csv
Test : test_set_scaled.csv
Shape: (100000, 5, 4, 1)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 5, 4, 16)	80
batch_normalization (BatchNormalization)	(None, 5, 4, 16)	64
conv2d_1 (Conv2D)	(None, 5, 4, 32)	2,080
batch_normalization_1 (BatchNormalization)	(None, 5, 4, 32)	128
max_pooling2d (MaxPooling2D)	(None, 2, 2, 32)	0
dropout (Dropout)	(None, 2, 2, 32)	0
conv2d_2 (Conv2D)	(None, 2, 2, 64)	8,256
batch_normalization_2 (BatchNormalization)	(None, 2, 2, 64)	256
global_average_pooling2d (GlobalAveragePooling2D)	(None, 64)	0
dense (Dense)	(None, 64)	4,160
dropout_1 (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 32)	2,080
dropout_2 (Dropout)	(None, 32)	0
dense_2 (Dense)	(None, 10)	330

```
 Total params: 17,434 (68.10 KB)
 Trainable params: 17,210 (67.23 KB)
 Non-trainable params: 224 (896.00 B)
Epoch 1/80
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 15s 7ms/step - accuracy: 0.7346 - loss: 1.3433 - val_accuracy: 0.9187 - val_loss
Epoch 2/80
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.9136 - loss: 0.9464 - val_accuracy: 0.9563 - val_loss
Epoch 3/80
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.9432 - loss: 0.8478 - val_accuracy: 0.9438 - val_loss
Epoch 4/80
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.9572 - loss: 0.7944 - val_accuracy: 0.9625 - val_loss
Epoch 5/80
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.9630 - loss: 0.7637 - val_accuracy: 0.9594 - val_loss
Epoch 6/80
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.9674 - loss: 0.7430 - val_accuracy: 0.9594 - val_loss
Epoch 7/80
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 20s 7ms/step - accuracy: 0.9696 - loss: 0.7297 - val_accuracy: 0.9656 - val_loss
Epoch 8/80
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.9700 - loss: 0.7234 - val_accuracy: 0.9625 - val_loss
Epoch 9/80
```

Ablation Study

```
# ============================================================
# ABLATION STUDY: 2D CNN FOR BEARING FAULT DETECTION
#   - Conv2D(16/32/64, (2,2), padding='same', l2(1e-3)) x3
#   - BatchNorm after every conv
#   - MaxPooling2D((2,2)) + Dropout(0.4) after the 2nd conv block
#   - GlobalAveragePooling2D
#   - Dense(64, l2(1e-3)) -> Dropout(0.5)
#   - Dense(32, l2(1e-3)) -> Dropout(0.4)
#   - Dense(num_classes, softmax)
#   - Adam(lr=5e-4), CategoricalCrossentropy(label_smoothing=0.1)
#   - EarlyStopping(patience=6), ReduceLROnPlateau(factor=0.3, patience=3)
#   - class_weight='balanced', batch_size=64, epochs=80

# !pip -q install tensorflow

import os
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.utils.class_weight import compute_class_weight
from tensorflow.keras.models import Model
```

```
from tensorflow.keras.layers import (
    Conv2D, BatchNormalization, MaxPooling2D, Dropout,
    GlobalAveragePooling2D, Flatten, Dense, Input
)
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.regularizers import l2

SEED = 42
np.random.seed(SEED)
tf.random.set_seed(SEED)

# ============================================================
# LOAD DATA
# ============================================================

def find_file(keyword):
    files_list = os.listdir()
    matches = [f for f in files_list if keyword in f.lower()]
    return matches[0]

train_file = find_file("augmented")
val_file = find_file("validation")
test_file = find_file("test")

print("Train:", train_file)
print("Val  :", val_file)
print("Test :", test_file)

train_df = pd.read_csv(train_file)
val_df = pd.read_csv(val_file)
test_df = pd.read_csv(test_file)

X_train_flat = train_df.iloc[:, :-1].values
y_train_raw = train_df.iloc[:, -1].values

X_val_flat = val_df.iloc[:, :-1].values
y_val_raw = val_df.iloc[:, -1].values

X_test_flat = test_df.iloc[:, :-1].values
y_test_raw = test_df.iloc[:, -1].values

classes = np.unique(y_train_raw)
weights = compute_class_weight(class_weight='balanced', classes=classes, y=y_train_raw)
class_weights = dict(zip(classes, weights))

y_train = to_categorical(y_train_raw)
y_val = to_categorical(y_val_raw)
y_test = to_categorical(y_test_raw)
num_classes = y_train.shape[1]

height = 5
width = 4
padding = (height * width) - X_train_flat.shape[1]

X_train = np.pad(X_train_flat, ((0, 0), (0, padding)), mode='constant')
X_val = np.pad(X_val_flat, ((0, 0), (0, padding)), mode='constant')
X_test = np.pad(X_test_flat, ((0, 0), (0, padding)), mode='constant')

X_train = X_train.reshape((X_train.shape[0], height, width, 1))
X_val = X_val.reshape((X_val.shape[0], height, width, 1))
X_test = X_test.reshape((X_test.shape[0], height, width, 1))

print("Shape:", X_train.shape, "| Classes:", num_classes)


# ============================================================
# CONFIGURABLE MODEL BUILDER
# ============================================================

DEFAULT_CONFIG = {
    "name": "0_baseline_exact",
    "conv_filters": [16, 32, 64],
    "kernel_size": (2, 2),
    "use_batchnorm": True,
    "use_dropout": True,
    "dropout_after_pool": 0.4,       # matches Dropout(0.4) after MaxPool
    "dropout_dense": [0.5, 0.4],     # matches : Dropout(0.5) after Dense(64), Dropout(0.4) after Dense(32)
    "use_l2": True,
    "l2_value": 1e-3,
    "use_maxpool": True,
```

```
    "pool_after_block": 1,           # pool after 2nd conv block (index 1),
    "pooling_type": "gap",           # "gap" or "flatten"
    "use_dense_head": True,
    "dense_units": [64, 32],
    "use_class_weights": True,
    "learning_rate": 5e-4,
    "use_lr_schedule": True,
    "lr_factor": 0.3,
    "lr_patience": 3,
    "early_stop_patience": 6,
    "label_smoothing": 0.1,
    "metric_average": "macro",
}


def reg(cfg):
    return l2(cfg["l2_value"]) if cfg["use_l2"] else None


def build_model(cfg):
    inputs = Input(shape=(height, width, 1))
    x = inputs

    for i, filters in enumerate(cfg["conv_filters"]):
        x = Conv2D(filters, cfg["kernel_size"], padding='same',
                   activation='relu', kernel_regularizer=reg(cfg))(x)
        if cfg["use_batchnorm"]:
            x = BatchNormalization()(x)
        if cfg["use_maxpool"] and i == cfg["pool_after_block"]:
            x = MaxPooling2D((2, 2))(x)
        if cfg["use_dropout"] and i == cfg["pool_after_block"]:
            x = Dropout(cfg["dropout_after_pool"])(x)

    if cfg["pooling_type"] == "gap":
        x = GlobalAveragePooling2D()(x)
    else:
        x = Flatten()(x)

    if cfg["use_dense_head"]:
        for units, drop_rate in zip(cfg["dense_units"], cfg["dropout_dense"]):
            x = Dense(units, activation='relu', kernel_regularizer=reg(cfg))(x)
            if cfg["use_dropout"]:
                x = Dropout(drop_rate)(x)

    outputs = Dense(num_classes, activation='softmax')(x)

    model = Model(inputs, outputs)
    loss = tf.keras.losses.CategoricalCrossentropy(label_smoothing=cfg["label_smoothing"])
    model.compile(
        optimizer=Adam(learning_rate=cfg["learning_rate"]),
        loss=loss,
        metrics=['accuracy'],
    )
    return model


def get_callbacks(cfg):
    callbacks = [EarlyStopping(monitor='val_loss', patience=cfg["early_stop_patience"],
                               restore_best_weights=True)]
    if cfg["use_lr_schedule"]:
        callbacks.append(ReduceLROnPlateau(monitor='val_loss', factor=cfg["lr_factor"],
                                           patience=cfg["lr_patience"], min_lr=1e-6))
    return callbacks


def run_experiment(cfg, epochs=80, batch_size=64, verbose=0):
    tf.keras.backend.clear_session()
    tf.random.set_seed(SEED)

    model = build_model(cfg)
    cw = class_weights if cfg["use_class_weights"] else None

    history = model.fit(
        X_train, y_train,
        epochs=epochs,
        batch_size=batch_size,
        validation_data=(X_val, y_val),
        callbacks=get_callbacks(cfg),
        class_weight=cw,
        verbose=verbose,
    )
```

```
    y_pred = model.predict(X_test, verbose=0)
    y_pred_classes = np.argmax(y_pred, axis=1)
    avg = cfg["metric_average"]

    metrics = {
        "variant": cfg["name"],
        "accuracy": accuracy_score(y_test_raw, y_pred_classes),
        "precision": precision_score(y_test_raw, y_pred_classes, average=avg, zero_division=0),
        "recall": recall_score(y_test_raw, y_pred_classes, average=avg, zero_division=0),
        "f1_score": f1_score(y_test_raw, y_pred_classes, average=avg, zero_division=0),
        "epochs_run": len(history.history['accuracy']),
        "final_val_accuracy": history.history['val_accuracy'][-1],
        "final_val_loss": history.history['val_loss'][-1],
        "trainable_params": model.count_params(),
    }
    return metrics, history


# ============================================================
# DEFINE ABLATION VARIANTS
# ============================================================

def make_cfg(overrides):
    cfg = dict(DEFAULT_CONFIG)
    cfg.update(overrides)
    return cfg


ablation_variants = [
    make_cfg({"name": "0_baseline_exact"}),

    make_cfg({"name": "1_no_batchnorm", "use_batchnorm": False}),

    make_cfg({"name": "2_no_dropout", "use_dropout": False}),

    make_cfg({"name": "3_no_l2_regularization", "use_l2": False}),

    make_cfg({"name": "4_flatten_instead_of_gap", "pooling_type": "flatten"}),

    make_cfg({
        "name": "5_two_conv_blocks_instead_of_three",
        "conv_filters": [16, 32],
        "pool_after_block": 1,
    }),

    make_cfg({"name": "6_larger_3x3_kernels", "kernel_size": (3, 3)}),

    make_cfg({"name": "7_no_maxpooling", "use_maxpool": False}),

    make_cfg({"name": "8_no_dense_head", "use_dense_head": False}),

    make_cfg({"name": "9_no_label_smoothing", "label_smoothing": 0.0}),

    make_cfg({
        "name": "10_fixed_higher_lr_no_schedule",
        "learning_rate": 2.5e-3,
        "use_lr_schedule": False,
    }),
]


# ============================================================
# RUN ALL EXPERIMENTS
# ============================================================
# NOTE: epochs=80 matches the original script but will make 11 full
# training runs slow. Consider epochs=30-40 for a faster first pass;
# EarlyStopping will still cut short unproductive runs either way.

EPOCHS = 40  # reduce from the original 80 for a faster ablation sweep;
             # bump back to 80 for a final, publication-grade run

results = []
histories = {}

for cfg in ablation_variants:
    print("\n" + "=" * 70)
    print(f"Running variant: {cfg['name']}")
    print("=" * 70)
    metrics, history = run_experiment(cfg, epochs=EPOCHS, batch_size=64, verbose=1)
    results.append(metrics)
    histories[cfg["name"]] = history.history
    print(f"  -> Accuracy: {metrics['accuracy']:.4f} | F1: {metrics['f1_score']:.4f} "
```

```
          f"| Epochs: {metrics['epochs_run']} | Params: {metrics['trainable_params']:,}")

results_df = pd.DataFrame(results).sort_values("f1_score", ascending=False).reset_index(drop=True)

print("\n" + "=" * 70)
print("ABLATION STUDY RESULTS (sorted by F1 score, macro-averaged)")
print("=" * 70)
print(results_df.to_string(index=False))

results_df.to_csv("cnn_ablation_results.csv", index=False)
with open("cnn_ablation_histories.json", "w") as f:
    json.dump(histories, f)

print("\nSaved: cnn_ablation_results.csv, cnn_ablation_histories.json")


# ============================================================
# VISUALIZATIONS
# ============================================================

baseline_row = results_df[results_df["variant"] == "0_baseline_exact"].iloc[0]

plt.figure(figsize=(14, 6))
metrics_to_plot = ["accuracy", "precision", "recall", "f1_score"]
x = np.arange(len(results_df))
width = 0.2
for i, metric in enumerate(metrics_to_plot):
    plt.bar(x + i * width, results_df[metric], width, label=metric)
plt.xticks(x + width * 1.5, results_df["variant"], rotation=45, ha='right')
plt.ylabel("Score")
plt.title("CNN Ablation Study: Metric Comparison Across Variants", fontsize=14, fontweight='bold')
plt.legend()
plt.grid(True, axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig("cnn_ablation_metric_comparison.png", dpi=150)
plt.show()

plt.figure(figsize=(12, 6))
delta_f1 = results_df["f1_score"] - baseline_row["f1_score"]
colors = ['#d62728' if v < 0 else '#2ca02c' for v in delta_f1]
plt.barh(results_df["variant"], delta_f1, color=colors)
plt.axvline(0, color='black', linewidth=0.8)
plt.xlabel("Change in F1 Score vs. Baseline")
plt.title("CNN: Impact of Removing/Changing Each Component (relative to baseline)",
          fontsize=13, fontweight='bold')
plt.grid(True, axis='x', alpha=0.3)
plt.tight_layout()
plt.savefig("cnn_ablation_delta_from_baseline.png", dpi=150)
plt.show()

plt.figure(figsize=(14, 6))
for name, hist in histories.items():
    plt.plot(hist['val_accuracy'], label=name, alpha=0.8)
plt.title("CNN Validation Accuracy Curves — All Variants", fontsize=14, fontweight='bold')
plt.xlabel("Epoch")
plt.ylabel("Validation Accuracy")
plt.legend(fontsize=8, loc='lower right')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("cnn_ablation_val_accuracy_curves.png", dpi=150)
plt.show()

print("\nSaved plots: cnn_ablation_metric_comparison.png, "
      "cnn_ablation_delta_from_baseline.png, cnn_ablation_val_accuracy_curves.png")

# ============================================================
# SUMMARY
# ============================================================
print("\n" + "=" * 70)
print("SUMMARY: Component impact (F1 delta vs baseline, macro-averaged)")
print("=" * 70)
summary = results_df.copy()
summary["f1_delta_vs_baseline"] = summary["f1_score"] - baseline_row["f1_score"]
summary = summary[summary["variant"] != "0_baseline_exact"]
summary = summary.sort_values("f1_delta_vs_baseline")
print(summary[["variant", "f1_delta_vs_baseline", "trainable_params"]].to_string(index=False))
```

```
Train: preprocessed_augmented_dataset_scaled.csv
Val  : validation_set_scaled.csv
Test : test_set_scaled.csv
Shape: (100000, 5, 4, 1) | Classes: 10

======================================================================
Running variant: 0_baseline_exact
======================================================================
Epoch 1/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 18s 8ms/step - accuracy: 0.7331 - loss: 1.3440 - val_accuracy: 0.9375 - val_loss:
Epoch 2/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.9189 - loss: 0.9407 - val_accuracy: 0.9500 - val_loss:
Epoch 3/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.9468 - loss: 0.8443 - val_accuracy: 0.9656 - val_loss:
Epoch 4/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 10s 7ms/step - accuracy: 0.9573 - loss: 0.7972 - val_accuracy: 0.9594 - val_loss:
Epoch 5/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.9614 - loss: 0.7684 - val_accuracy: 0.9625 - val_loss:
Epoch 6/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.9663 - loss: 0.7462 - val_accuracy: 0.9594 - val_loss:
Epoch 7/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.9685 - loss: 0.7326 - val_accuracy: 0.9656 - val_loss:
Epoch 8/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.9705 - loss: 0.7232 - val_accuracy: 0.9625 - val_loss:
Epoch 9/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 12s 8ms/step - accuracy: 0.9711 - loss: 0.7166 - val_accuracy: 0.9656 - val_loss:
Epoch 10/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.9736 - loss: 0.7088 - val_accuracy: 0.9625 - val_loss:
Epoch 11/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.9733 - loss: 0.7071 - val_accuracy: 0.9656 - val_loss:
Epoch 12/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 21s 8ms/step - accuracy: 0.9751 - loss: 0.7028 - val_accuracy: 0.9625 - val_loss:
Epoch 13/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.9771 - loss: 0.6973 - val_accuracy: 0.9594 - val_loss:
Epoch 14/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.9769 - loss: 0.6957 - val_accuracy: 0.9594 - val_loss:
Epoch 15/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.9818 - loss: 0.6813 - val_accuracy: 0.9656 - val_loss:
Epoch 16/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 12s 8ms/step - accuracy: 0.9844 - loss: 0.6756 - val_accuracy: 0.9625 - val_loss:
Epoch 17/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 12s 8ms/step - accuracy: 0.9845 - loss: 0.6722 - val_accuracy: 0.9625 - val_loss:
  -> Accuracy: 0.9394 | F1: 0.9391 | Epochs: 17 | Params: 17,434

======================================================================
Running variant: 1_no_batchnorm
======================================================================
Epoch 1/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 13s 7ms/step - accuracy: 0.6458 - loss: 1.4692 - val_accuracy: 0.8500 - val_loss:
Epoch 2/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.8331 - loss: 1.1110 - val_accuracy: 0.8938 - val_loss:
Epoch 3/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.8683 - loss: 1.0259 - val_accuracy: 0.9125 - val_loss: 0
Epoch 4/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 6ms/step - accuracy: 0.8860 - loss: 0.9770 - val_accuracy: 0.9219 - val_loss:
Epoch 5/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.9011 - loss: 0.9473 - val_accuracy: 0.9250 - val_loss:
Epoch 6/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 9s 6ms/step - accuracy: 0.9105 - loss: 0.9271 - val_accuracy: 0.9438 - val_loss: 0
Epoch 7/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9194 - loss: 0.9112 - val_accuracy: 0.9438 - val_loss: 0
Epoch 8/40
```

Final 2D CNN Model

```
# ============================================================
# FINAL 2D CNN MODEL FOR BEARING FAULT DETECTION
# ============================================================

# !pip -q install tensorflow

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf

from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, roc_curve, auc
)
from sklearn.utils.class_weight import compute_class_weight
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (
    Conv2D, Dense, Dropout, Input, BatchNormalization,
    MaxPooling2D, Flatten
)
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
```

```
from tensorflow.keras.optimizers import Adam

from google.colab import files
uploaded = files.upload()

# ============================================================
# LOAD DATA
# ============================================================

files_list = os.listdir()

def find_file(keyword):
    matches = [f for f in files_list if keyword in f.lower()]
    return matches[0]

train_file = find_file("augmented")
val_file = find_file("validation")
test_file = find_file("test")

print("Train:", train_file)
print("Val  :", val_file)
print("Test :", test_file)

train_df = pd.read_csv(train_file)
val_df = pd.read_csv(val_file)
test_df = pd.read_csv(test_file)

X_train = train_df.iloc[:, :-1].values
y_train_raw = train_df.iloc[:, -1].values

X_val = val_df.iloc[:, :-1].values
y_val_raw = val_df.iloc[:, -1].values

X_test = test_df.iloc[:, :-1].values
y_test_raw = test_df.iloc[:, -1].values

classes = np.unique(y_train_raw)
weights = compute_class_weight(class_weight='balanced', classes=classes, y=y_train_raw)
class_weights = dict(zip(classes, weights))

y_train = to_categorical(y_train_raw)
y_val = to_categorical(y_val_raw)
y_test = to_categorical(y_test_raw)

# Reshape data for 2D CNN input
height = 5
width = 4
padding = (height * width) - X_train.shape[1]

X_train = np.pad(X_train, ((0, 0), (0, padding)), mode='constant')
X_val = np.pad(X_val, ((0, 0), (0, padding)), mode='constant')
X_test = np.pad(X_test, ((0, 0), (0, padding)), mode='constant')

X_train = X_train.reshape((X_train.shape[0], height, width, 1))
X_val = X_val.reshape((X_val.shape[0], height, width, 1))
X_test = X_test.reshape((X_test.shape[0], height, width, 1))

print("Shape:", X_train.shape)

# ============================================================
# FINAL CNN MODEL
# ============================================================

model = Sequential([
    Input(shape=(height, width, 1)),

    Conv2D(16, (2, 2), padding='same', activation='relu'),
    BatchNormalization(),

    Conv2D(32, (2, 2), padding='same', activation='relu'),
    BatchNormalization(),

    MaxPooling2D((2, 2)),
    Dropout(0.4),

    Conv2D(64, (2, 2), padding='same', activation='relu'),
    BatchNormalization(),

    Flatten(),

    Dense(64, activation='relu'),
    Dropout(0.5),
```

```
    Dense(32, activation='relu'),
    Dropout(0.4),

    Dense(y_train.shape[1], activation='softmax')
])

# ============================================================
# COMPILE
# ============================================================

model.compile(
    optimizer=Adam(learning_rate=0.0005),
    loss=tf.keras.losses.CategoricalCrossentropy(label_smoothing=0.1),
    metrics=['accuracy']
)

model.summary()

# ============================================================
# CALLBACKS
# ============================================================

early_stop = EarlyStopping(
    monitor='val_loss',
    patience=6,
    restore_best_weights=True
)

reduce_lr = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.3,
    patience=3,
    min_lr=1e-6,
    verbose=1
)

# ============================================================
# TRAIN FINAL MODEL
# ============================================================

history = model.fit(
    X_train,
    y_train,
    epochs=80,
    batch_size=64,
    validation_data=(X_val, y_val),
    callbacks=[early_stop, reduce_lr],
    class_weight=class_weights,
    verbose=1
)

# ============================================================
# EVALUATION
# ============================================================

y_pred = model.predict(X_test)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true = y_test_raw

accuracy = accuracy_score(y_true, y_pred_classes)
precision = precision_score(y_true, y_pred_classes, average='macro')
recall = recall_score(y_true, y_pred_classes, average='macro')
f1 = f1_score(y_true, y_pred_classes, average='macro')

print("\nFINAL 2D CNN PERFORMANCE")
print(f"Accuracy  : {accuracy:.4f}")
print(f"Precision : {precision:.4f}")
print(f"Recall    : {recall:.4f}")
print(f"F1 Score  : {f1:.4f}")

# ============================================================
# CONFUSION MATRIX (PLOT + PRINT)
# ============================================================

cm = confusion_matrix(y_true, y_pred_classes)

print("\nConfusion Matrix (counts):")
print(cm)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=classes, yticklabels=classes)
plt.title("Final CNN Confusion Matrix")
```

```
plt.ylabel("True Label")
plt.xlabel("Predicted Label")
plt.tight_layout()
plt.savefig("final_cnn_confusion_matrix.png", dpi=150)
plt.show()

print("\nClassification Report:")
print(classification_report(y_true, y_pred_classes, target_names=[str(c) for c in classes]))

# ============================================================
# ROC CURVE
# ============================================================

plt.figure(figsize=(8, 6))
for i in range(y_test.shape[1]):
    fpr, tpr, _ = roc_curve(y_test[:, i], y_pred[:, i])
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, label=f'Class {i} (AUC={roc_auc:.2f})')
plt.plot([0, 1], [0, 1], 'k--')
plt.title("Final CNN ROC Curve")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend()
plt.tight_layout()
plt.savefig("final_cnn_roc_curve.png", dpi=150)
plt.show()

# ============================================================
# TRAINING CURVES
# ============================================================

plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train')
plt.plot(history.history['val_accuracy'], label='Val')
plt.title("Final CNN Accuracy")
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train')
plt.plot(history.history['val_loss'], label='Val')
plt.title("Final CNN Loss")
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("final_cnn_curves.png", dpi=150)
plt.show()

# ============================================================
# SAVE FINAL MODEL
# ============================================================

model.save("cnn_2d_model_final.keras")
print("\nFinal model saved to: cnn_2d_model_final.keras")
```

```
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 16s 10ms/step - accuracy: 0.9743 - loss: 0.7090 - val_accuracy: 0.9500 - val_loss:
Epoch 12/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 16s 10ms/step - accuracy: 0.9745 - loss: 0.7077 - val_accuracy: 0.9656 - val_loss:
Epoch 13/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 16s 10ms/step - accuracy: 0.9803 - loss: 0.6919 - val_accuracy: 0.9625 - val_loss:
Epoch 14/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 18s 11ms/step - accuracy: 0.9813 - loss: 0.6875 - val_accuracy: 0.9656 - val_loss:
Epoch 15/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 16s 10ms/step - accuracy: 0.9822 - loss: 0.6839 - val_accuracy: 0.9625 - val_loss:
  -> Accuracy: 0.9424 | F1: 0.9439 | Epochs: 15 | Params: 17,434

======================================================================
Running variant: 8_no_dense_head
======================================================================
Epoch 1/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 15s 8ms/step - accuracy: 0.9062 - loss: 0.8408 - val_accuracy: 0.9531 - val_loss:
Epoch 2/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 12s 8ms/step - accuracy: 0.9647 - loss: 0.6669 - val_accuracy: 0.9656 - val_loss:
Epoch 3/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.9750 - loss: 0.6263 - val_accuracy: 0.9563 - val_loss:
Epoch 4/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.9804 - loss: 0.6082 - val_accuracy: 0.9594 - val_loss:
Epoch 5/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.9836 - loss: 0.5962 - val_accuracy: 0.9531 - val_loss:
Epoch 6/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.9855 - loss: 0.5892 - val_accuracy: 0.9594 - val_loss:
Epoch 7/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 12s 8ms/step - accuracy: 0.9869 - loss: 0.5838 - val_accuracy: 0.9594 - val_loss:
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 12s 8ms/step - accuracy: 0.9883 - loss: 0.5797 - val_accuracy: 0.9625 - val_loss:
Epoch 9/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.9892 - loss: 0.5756 - val_accuracy: 0.9500 - val_loss:
Epoch 10/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.9894 - loss: 0.5735 - val_accuracy: 0.9594 - val_loss:
Epoch 11/40
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.9905 - loss: 0.5695 - val_accuracy: 0.9656 - val_loss:
```

Saving test_set_scaled.csv to test_set_scaled (1).csv
Saving validation_set_scaled.csv to validation_set_scaled (1).csv
Saving preprocessed_augmented_dataset_scaled.csv to preprocessed_augmented_dataset_scaled (1).csv
Train: preprocessed_augmented_dataset_scaled.csv
Val: validation_set_scaled.csv